

beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in **beamer** documents. Named overlay specifications are very handy both during edition and to manage complex and variable **beamer** overlay specifications. In particular, they allow to replace raw numbers in **beamer** `<...>` overlay specifications by logical identifiers. Demonstration files are [available for download](#) as part of the [development repository](#). As a side effect, this is another solution to this [latex.org forum query](#).

Contents

1	Implementation	1
1.1	Package declarations	1
1.2	Overview	2
1.3	Main scanner	2
1.3.1	Close branch	2
1.3.2	Control functions	3
1.3.3	Control functions	4
1.3.4	Delimiters	7
1.3.5	Raw units	8
1.3.6	Comma separated lists	10
1.3.7	ISR unit	13
1.3.8	<code>\Rnd</code> , <code>\Rslv</code> and <code>\ISRC</code> units	18
1.3.9	Call and assignment units	20

1 Implementation

1.1 Package declarations

```
1 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
2 \ProvidesExplPackage
3   {beanoves}
4   {2025/06/03}
5   {1.0}
6   {Named overlay specifications for beamer}
```

1.2 Overview

Reserved namespace: identifiers containing the case insensitive string **beanoves** or containing the case insensitive string **bnvs** delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into **beamer** specifications based on this data model.

The development is made easier due to a facility layer over `expl`.

```

7 \regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
8   \A \s* (?
    • 2 → V
9     ( [^:]+? )
    • 3, 4, 5 → A : Z? or A :: L?
10    | (? ( [^:]+? ) \s* : (? \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )
    • 6, 7 → ::(L:Z)?
11    | (? :: \s* (? ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
    • 8, 9 → :(Z::L)?
12    | (? : \s* (? ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
13  )
14  \s* \Z
15 }
```

1.3 Main scanner

1.3.1 Close branch

```

\c__bnvs_Cr1_close_regex  Curly closing delimiter.
16 \regex_const:Nn \c__bnvs_Cr1_close_regex { \s* %{
17   \} \s* }
```

(End of definition for \c__bnvs_Cr1_close_regex.)

```

\c__bnvs_close_regex  Closing delimiters.
18 \regex_const:Nn \c__bnvs_close_regex { \s* (%([{
19   \}|\)|\)) \s* }
```

(End of definition for `\c__bnvs_close_regex`.)

<code>\BNVS_scan_close:Fw</code>	<code>\BNVS_scan_close:Fw {<i><else:></i>} <i><input stream></i></code>
<code>\BNVS_scanned_close:Nw</code>	<code>\BNVS_scanned_close:Nw <i><clositer></i> <i><input stream></i></code>
<code>\BNVS_scanned_close_set:</code>	<code>\BNVS_scanned_close:w <i><input stream></i></code>
<code>\BNVS_scanned_close:w</code>	<code>\BNVS_scanned_close_set:n <i><close:w></i></code>

When a closing delimiter is scanned, call `\BNVS_scanned_close:w` with that delimiter as argument. Otherwise execute *<else:>* code.

`\BNVS_scanned_close:w` is a branch function called when the closing delimiter *<clositer>* has just been scanned. After it manages eventual errors in balancing delimiters, it calls the branch function `\BNVS_scanned_close:w` used many times. When redefined (for example by `\__bnvs_scan_delimited:NNnFw`), the `\BNVS_scanned_close:w` function does not forward to the eponym function, instead it must know where to go next.

```

20 \cs_new:Npn \BNVS_scan_close:Fw #1 {
21   \peek_regex_replace_once:NnTF \c__bnvs_close_regex { \1 } {
22     \BNVS_scanned_close:Nw
23   } {
24     #1
25   }
26 }

27 \cs_new:Npn \BNVS_scanned_close:Nw #1 {
28   \BNVS_error:n { Unexpected~delimiter~`#1' }
29   \BNVS_scanned_close:w
30 }

31 \tl_map_inline:nn {{close}{comma}{one_colon}{colons}{alter}} {
32   \cs_new:cpn { BNVS_scanned_#1_set:n } ##1 {
33     \cs_set:cpn { BNVS_scanned_#1:w } {
34       ##1
35     }
36   }

```

¶ Do nothing operation at the top level.

```

37   \cs_new:cpn { BNVS_scanned_#1:w } {
38     }
39 }

```

1.3.2 Control functions

<code>\BNVS_scan:nw</code>	<code>\BNVS_scan:w <i><input stream></i></code>
<code>\BNVS_scan:w</code>	<code>\BNVS_scan:nw {<i><whatever></i>} <i><input stream></i></code>

Medley of the above,

```

40 \cs_new:Npn \BNVS_scan:w {
41   \BNVS_scan_close:Fw {
42     \BNVS_scan_colon:Fw {
43       \BNVS_scan_stop:F {
44         \BNVS_scan_alter:Fw {
45           \BNVS_scan_comma:Fw {
46             \BNVS_scan:nw
47           }
48         }
49       }
50     }
51   }
52 }

53 \cs_new:Npn \BNVS_scan:nw #1 {
54   \BNVS_xprt_store:n { #1 }
55   \BNVS_scan:w
56 }

```

```
\BNVS_scan_delimiter:Fw \BNVS_scan_delimiter:Fw {\else:} \langle input stream \rangle
```

Medley of the above,

```

57 \cs_new:Npn \BNVS_scan_delimiter:Fw #1 {
58   \BNVS_scan_close:Fw {
59     \BNVS_scan_comma:Fw {
60       \BNVS_scan_colon:Fw {
61         \BNVS_scan_stop:F {
62           #1
63         }
64       }
65     }
66   }
67 }

```

1.3.3 Control functions

```
\BNVS_scan:nnnnnnnnnw \BNVS_scan:nnnnnnnnnw {\preamble:}
{\closed:} {\comma:} {\one_colon:} {\colons:} {\stopped:} {\alter:}
{\scan:w} \langle input stream \rangle
```

Start a scanning branch section by executing the `\preamble:` instructions. The six next arguments are function bodies used to define the corresponding branch functions. The `\scan:w` instructions are finally executed. They are expected to scan the input stream and terminate by calling one of the branch functions just defined. They consist of some `..._scan_...` function defined below.

```

68 \cs_new:Npn \BNVS_scan:nnnnnnnnnw #1 #2 #3 #4 #5 #6 #7 #8 {
69   #1
70   \BNVS_scanned_close_set:n { #2 }
71   \BNVS_scanned_comma_set:n { #3 }
72   \BNVS_scanned_one_colon_set:n { #4 }
73   \BNVS_scanned_colons_set:n { #5 }

```

```

74 \BNVS_scanned_stop_set:n { #6 }
75 \BNVS_scanned_alter_set:n { #7 }
76 #8

```

⊗ There must be absolutely no code here. No hooking allowed either.

```

77 }

```

`\BNVS_begin_scan:` `\BNVS_begin_scan:`

Start a scanning group.

```

78 \cs_new:Npn \BNVS_begin_scan: {
79   \BNVS_scan_will_begin:
80   \BNVS_begin:
81   \BNVS_scan_did_begin:
82 }

83 \cs_new:Npn \BNVS_scan_will_begin: {
84   \BNVS_xprt_stored:
85 }

86 \cs_new:Npn \BNVS_scan_did_begin: {
87   \BNVS_xprt_clear:
88   \BNVS_xprt_wrap_prefix:n {}
89   \BNVS_next_set:n {}
90 }

```

`\BNVS_end_scan:n` `\BNVS_end_scan:n {⟨material-x⟩}`
`\BNVS_end_scan:` `\BNVS_end_scan:`
`\BNVS_end_scan_next:` `\BNVS_end_scan_no_I:`

`\BNVS_end_scan_next:`

One of these functions must be called to close the TeX scan group. `\BNVS_end_scan_next:` calls `\BNVS_end_scan:` and what was stored as next instructions before closing the group. `\BNVS_end_scan:` call `\BNVS_end_scan:n` on what was exported before closing the group (at this level), once prepared by `\BNVS_xprt_wrapped:n...` Notice that `⟨material-x⟩` is x-expanded.

```

91 \cs_new:Npn \BNVS_end_scan:n #1 {
92   \__bnvs_tl_if_empty:cTF { I } {
93     \exp_args:NNx \BNVS_end: \BNVS_xprt:n {
94       \BNVS_xprt_wrapped:n { #1 }
95     }
96   } {
97     \BNVS_tl_use:nv {
98       \exp_args:Nx \BNVS_end_scan:nnn {
99         \BNVS_xprt_wrapped:n { #1 }
100       }
101     } { I } { S ( \l__bnvs_I_tl ) }
102   }
103 }

104 \cs_new:Npn \BNVS_end_scan:nnn #1 #2 #3 {
105   \BNVS_end:
106   \BNVS_xprt:n { #1 }
107   \__bnvs_tl_set:cn I { #2 }
108   \__bnvs_tl_set:cn { S ( #2 ) } { #3 }
109 }

```

```

110 \cs_new:Npn \BNVS_end_scan: {
111   \BNVS_xprt_use:n {
112     \BNVS_end_scan:n
113   }
114 }

115 \cs_new:Npn \BNVS_end_scan_next: {
116   \BNVS_next_use:n { \BNVS_end_scan: }
117 }

```

```

\BNVS_begin_scan:nn \BNVS_begin_scan:nn {<xprt:n>} {<next:>}

```

 **OBSOLETE.** Call `\BNVS_begin_scan:` with default arguments that end the scan group and call the eponym branch function.

```

118 \cs_new:Npn \BNVS_begin_scan:nn #1 #2 {
119   \BNVS_begin_scan:
120   \BNVS_xprt_wrap:n { #1 }
121   \BNVS_next_set:n { #2 }
122 }

```

```

\BNVS_scan:nnw \BNVS_scan:nnw {<preamble:>} {<scan:w>} <input stream>

```

Start a scanning branch section by executing the `<preamble:>` instructions, setting the branch function to default values and executing the `<scan:w>` instructions.

```

123 \cs_new:Npn \BNVS_scan:nnw #1 #2 {
124   \BNVS_scan:nnnnnnnnw {
125     \BNVS_begin_scan:
126     #1
127   } {
128     \BNVS_end_scan: \BNVS_scanned_close:w
129   } {
130     \BNVS_end_scan: \BNVS_scanned_comma:w
131   } {
132     \BNVS_end_scan: \BNVS_scanned_one_colon:w
133   } {
134     \BNVS_end_scan: \BNVS_scanned_colons:w
135   } {
136     \BNVS_end_scan: \BNVS_scanned_stop:
137   } {
138     \BNVS_end_scan: \BNVS_scanned_alter:w
139   } {
140     #2
141   }
142 }

```

1.3.4 Delimiters

We assume that delimiters are paired and balanced. Nevertheless, some errors are caught and properly managed.

\BNVS_scan_close:nnnNnw	\BNVS_scan_close:nnnNnw {⟨preamble:⟩} {⟨close:w⟩} {⟨stop:⟩} ⟨close⟩ {⟨scan:w⟩}
\BNVS_scan_close:nnnnNnw	\BNVS_scan_close:nnnnNnw {⟨xprt prefix:w⟩} {⟨xprt wrap:n⟩} {⟨close:w⟩} {⟨stop:⟩}

An opening delimiter has been scanned, scan the material up to the balancing closing delimiter using `⟨scan:w⟩`. Implementation detail: we begin a scanning group that will be closed on scanning the `⟨close⟩` delimiter, or a different closing delimiter, always at the same level. The `⟨scan:w⟩` function scans the `⟨input stream⟩`, and is expected to end on a closing delimiter at the current level by sending `\BNVS_scanned_close:w`. This is the normal instruction flow.

```

143 \cs_new:Npn \BNVS_scan_close:nnnNnw #1 #2 #3 #4 #5 {
144   \BNVS_scan:nnnnnnnw {
❧ One group level for the contents.
145     \BNVS_begin_scan:
146     \cs_set:Npn \BNVS_scanned_close:Nw ##1 {
147       \tl_if_eq:nnF { #4 } { ##1 } {
❧ If the closing delimiter scanned is not the expected one, raise.
148         \BNVS_error:n { Unexpected~`##1'~instead~of~`#4'. }
149       }
❧ Anyways, branch here.
150       \BNVS_scanned_close:w
151     }
152     #1
153   } {
❧ Close branch. End the group and execute ⟨close:w⟩ instructions. Intermediate groups
should be closed appropriately with the right definition of branch functions. Clearing
the I variable before returning has the side effect not to export the actual value of I and
keep what is already in the containing group.
154   \__bnvs_tl_clear:c I
155   \BNVS_end_scan:
156   #2
157 } {
❧ Any other branch function raises and forwards to \BNVS_scanned_close:w,
158   \BNVS_error:n { Missing~`#4'. }
159   \BNVS_scanned_close:w
160 } {
161   \BNVS_error:n { Missing~`#4'. }
162   \BNVS_scanned_close:w
163 } {
164   \BNVS_error:n { Missing~`#4'. }
165   \BNVS_scanned_close:w
166 } {
❧ or \BNVS_scanned_stop: after the group ends.

```

```

167     \BNVS_error:n { Missing~`#4'. }
168     \BNVS_end_scan:
169     #3
170 } {
❗ Unreachable code?

171     \BNVS_error:n { Missing~`#4'. }
172     \BNVS_scanned_close:w
173 } {

❗ One group level for the contents.

174     #5
175 }
176 }

177 \cs_new:Npn \BNVS_scan_close:nnnnNnw #1 #2 {
178     \BNVS_scan_close:nnnNnw {
179         \BNVS_xprt_wrap_prefix:n { #1 }
180         \BNVS_xprt_wrap_raw:n { #2 }
181     }
182 }

```

1.3.5 Raw units

```

1  ⟨Call⟩    ::= '(' ⟨RawLst⟩ ')'
2  ⟨Raw⟩     ::= '"' ⟨Call⟩
3  ⟨RawLst⟩  ::= ( ⟨RawRnd⟩ | ⟨RawSqr⟩ | ⟨RawCrl⟩ | ⟨RawOtr⟩ ) *
4  ⟨RawRnd⟩  ::= '(' ⟨RawLst⟩ ')'
5  ⟨RawSqr⟩  ::= '[' ⟨RawLst⟩ ']'
6  ⟨RawCrl⟩  ::= '{' ⟨RawLst⟩ '}'
7  ⟨RawOtr⟩  ::= ⟨spc⟩ | ⟨comma⟩ | ⟨alnum⟩ | ⟨other⟩ | '.' | '?' | ''

```

`\c__bnvs_Rnd_open_regex` Round opening delimiter.

```

183 \regex_const:Nn \c__bnvs_Rnd_open_regex { \s* \( \%
184     \s* }

```

(End of definition for \c__bnvs_Rnd_open_regex.)

`\c__bnvs_Sqr_open_regex` Square opening delimiter.

```

185 \regex_const:Nn \c__bnvs_Sqr_open_regex { \s* \[ \%
186     \s* }

```

(End of definition for \c__bnvs_Sqr_open_regex.)

`\c__bnvs_Crl_open_regex` Curly opening delimiter.

```

187 \regex_const:Nn \c__bnvs_Crl_open_regex { \s* \{ \%
188     \s* }

```

(End of definition for \c__bnvs_Crl_open_regex.)

`\c__bnvs_open_raw_regex` Raw opening delimiter.


```

189 \regex_const:Nn \c__bnvs_open_raw_regex { \s* "\(" %)
190 \s* }

```

(End of definition for \c__bnvs_open_raw_regex.)

```

\BNVS_scan_raw:w \BNVS_scan_raw:w <input stream>

```

Scan balanced material taking only delimiters into account. Used to parse calls to functions.

```

191 \cs_new:Npn \BNVS_scan_raw:n #1 {
192   \BNVS_xprt:n { #1 }
193   \BNVS_scan_raw:w
194 }

195 \group_begin:
196 \char_set_catcode_group_begin:N <
197 \char_set_catcode_group_end:N >
198 \char_set_catcode_other:N \{
199 \char_set_catcode_other:N \}
200 \cs_new:Npn \BNVS_scan_raw:w <
201   \peek_regex_replace_once:NnTF \c__bnvs_Rnd_open_regex < > <
202     \BNVS_scan_close:nnnNnw
203     < \BNVS_xprt_space_set:n < \BNVS_xprt:n < ~ > >
204     \BNVS_xprt_wrap_raw:n < ##1 > >%(
205     < \BNVS_xprt:n < ) > \BNVS_scan_raw:w >%(
206     < \BNVS_error:n < Missing~`)' > %(
207     \BNVS_xprt:n < ) > \BNVS_scan_stop: > %(
208     ) < \BNVS_xprt:n < ( > \BNVS_scan_raw:w >%(
209   > <
210   \peek_regex_replace_once:NnTF \c__bnvs_Sqr_open_regex < > <
211     \BNVS_scan_close:nnnNnw
212     < \BNVS_xprt_space_set:n < \BNVS_xprt:n < ~ > >
213     \BNVS_xprt_wrap_raw:n < ##1 > >%(
214     < \BNVS_xprt:n < ] > \BNVS_scan_raw:w >%(
215     < \BNVS_error:n < Missing~`] ' > %(
216     \BNVS_xprt:n < ] > \BNVS_scan_raw:w >%(
217     ] < \BNVS_xprt:n < [ > \BNVS_scan_raw:w >%(
218   > <
219   \peek_regex_replace_once:NnTF \c__bnvs_Crl_open_regex < > <
220     \BNVS_scan_close:nnnNnw
221     < \BNVS_xprt_space_set:n < \BNVS_xprt:n < ~ > >
222     \BNVS_xprt_wrap_raw:n < ##1 > >%(
223     < \BNVS_xprt:n < } > \BNVS_scan_raw:w >%(
224     < \BNVS_error:n < Missing~`}' > %(
225     \BNVS_xprt:n < } > \BNVS_scan_raw:w >%(
226     } < \BNVS_xprt:n < { > \BNVS_scan_raw:w >%(
227   > <
228   \BNVS_scan_close:Fw <
229   \BNVS_scan_stop:F <
230   \BNVS_scan_raw:n
231   >
232   >

```

```

233     >
234     >
235     >
236     >
237 \group_end:

```

```

\BNVS_scan_raw:TTFw \BNVS_raw:TTFw {\close:w} {\stop:} \input stream
\BNVS_scan_call:TTFw \BNVS_call:TTFw {\close:w} {\stop:} \input stream

```

Scan balanced material taking only delimiters into account. Used to parse calls to functions. An "(character combination has just been scanned.

```

238 \cs_new:Npn \BNVS_scan_raw:TTFw #1 #2 #3 {
239   \peek_regex_replace_once:NnTF \c__bnvs_open_raw_regex { } {
240     \BNVS_scan_close:nnnNnw {
241       \BNVS_xprt_wrap_raw:n { \:n { ##1 } }
242       \BNVS_xprt_space_set:n { \BNVS_xprt:n { ~ } }
243     } { #1 } { #2 } %(
244     ) {
245       \BNVS_scan_raw:w
246     }
247   } {
248     #3
249   }
250 }

```

1.3.6 Comma separated lists

Here is a partial grammar for a comma separated list template.

$$\langle \text{csl}(\langle \text{item} \rangle) \rangle ::= \langle \text{item} \rangle ? (\langle \text{comma} \rangle \langle \text{item} \rangle)^*$$

The corresponding AST template is below, where $\langle \text{item} \rangle^*$ is the AST for $\langle \text{item} \rangle$:

$$\backslash:nn \{ \langle \text{item} \rangle^* \} \{ \backslash:nn \{ \langle \text{item} \rangle^* \} \{ \dots \} \backslash:nn \{ \langle \text{item} \rangle^* \} \{ \} \{ \dots \} \} \}$$

```

\BNVS_scan_csl:nnnnw \BNVS_scan_csl:nnnnw {\preamble:} {\close:w} {\stop:} {\scan:w} \input stream

```

This function scans the comma separated list of items, regardless the enclosing delimiters. It begins a scanning TeX group that is ended by a call to `\BNVS_scanned_close:w`, executing $\langle \text{close:w} \rangle$, or `\BNVS_scanned_stop:`, executing $\langle \text{stop:} \rangle$. The scanned material is then exported between braces. On scanning a comma, a new group is created. The exported material is empty for an empty list, for input a it is `\:nn{a}{}`, for input a,b it is `\:nn{a}{\:nn{b}{}}`, and so on.

```

251 \cs_new:Npn \BNVS_scan_csl:nnnnw #1 #2 #3 #4 {
252   \BNVS_scan:nnnnnnnw {

```

¶ One group level for the contents.

```

253   \BNVS_begin_scan:

```

```

254     \BNVS_xprt_wrap_prefix:n { \:nn }
255     \BNVS_xprt_wrap:n { { ##1 } { } }

❧ Execute the <preamble:> instructions..

256     #1
257 }

❧ Close branch.

258     { \BNVS_end_scan: #2 }
259     {
260         \BNVS_xprt_use:n { \tl_if_empty:nTF } {

❧ Comma branch, the first item is empty, skip it.

261         #4
262     } {

❧ Comma branch, the first item is not empty, there will be another item, possibly empty.

263         \BNVS_xprt_wrap:n { { #####1 } }
264         \BNVS_end_scan:
265         \BNVS_scan:nnnnnnnnnw {
266             \BNVS_begin_scan:
267             \BNVS_xprt_wrap:n { { #####1 } }
268         }
269         { \BNVS_end_scan: \BNVS_scanned_close:w }
270         { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
271         { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
272         { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
273         { \BNVS_end_scan: \BNVS_scanned_stop: }
274         { #4 }
275     {
276         \BNVS_scan_csl:nnnw { } {

❧ Close branch.

277         \BNVS_if_xprted_empty:TF {
278             \BNVS_end_scan:
279             \BNVS_xprt:n { { } }
280         } {
281             \BNVS_end_scan:
282         }
283         \BNVS_scanned_close:w
284     } {

❧ Stop branch.

285         \BNVS_if_xprted_empty:TF {
286             \BNVS_end_scan:
287             \BNVS_xprt:n { { } }
288         } {
289             \BNVS_end_scan:
290         }
291         \BNVS_scanned_stop:
292     } {
293         #4
294     }
295 }
296 }

```

```

297     }
298     { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
299     { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
300     { \BNVS_end_scan: #3 }
301     { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }

```

¶ By default, the list contains only one item.

```

302     { #4 }
303 }

```

\BNVS_scan_csl_item:w **\BNVS_scan_csl_item:w** *<input stream>*

Scan a single item in a comma separated list. The item ends at the first top level comma, the first unbalanced delimiter if one is found, or when a **\q_stop** is found. The **stored** variable is fed with material that cannot be processed by **<scan:Fw>** function.

```

304 \cs_set:Npn \BNVS_scan_csl_item:w {
305   \BNVS_scan_comma:Fw {
306     \BNVS_scan_close:Fw {
307       \BNVS_scan_stop:F {
308         \BNVS_scan_csl_special:TFw {
309           \BNVS_scan_csl_item:w
310         } {
311           \__bnvs_scan_csl_other:Nw
312         }
313       }
314     }
315   }
316 }

317 \BNVS_new:cpn { scan_csl_other:Nw } #1 {
318   \BNVS_xprt_store:n { #1 }
319   \BNVS_scan_csl_item:w
320 }

```

\BNVS_scan_csl:nw **\BNVS_scan_csl:nw** *<preamble:>* *<input stream>*

These functions scan the comma separated list of items, regardless the enclosing delimiters. **<alter:TFw>** is the argument of **\BNVS_scan_alter_wrap:n**.

```

321 \cs_new:Npn \BNVS_scan_csl:nw #1 {
322   \BNVS_scan:nnnnnnnw {

```

¶ One group level for the contents.

```

323     \BNVS_begin_scan:
324     \BNVS_xprt_wrap_prefix:n { \:nn }
325     \BNVS_xprt_wrap:n { { ##1 } {} }
326     #1
327   } {

```

¶ Close branch.

```

328     \BNVS_end_scan: \BNVS_scanned_close:w
329   } {
330     \BNVS_if_xprted_empty:TF {

```

¶ Comma branch, the first item is empty, skip it.

```

331         \BNVS_scan_csl_item:w
332     } {

```

¶ Comma branch, the first item is not empty, there will be another item, possibly empty. Wrap what is already available between curly brackets and open a group.

```

333         \BNVS_xprt_use:n { \BNVS_xprt_clear: \BNVS_xprt_grp:n }
334         \BNVS_xprt_wrap:n { #####1 }
335         \BNVS_scan:nnnnnnnw {
336             \BNVS_begin_scan:
337             \BNVS_xprt_wrap_grp:n { #####1 }
338         }
339         { \BNVS_end_scan: \BNVS_scanned_close:w }
340         { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
341         { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
342         { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
343         { \BNVS_end_scan: \BNVS_scanned_stop: }
344         { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
345     {
346         \BNVS_scan_csl:nw { }
347     }
348 }
349 }
350 { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
351 { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }
352 { \BNVS_end_scan: \BNVS_scanned_stop: }
353 { \BNVS_error:n { Unreachable } \BNVS_scanned_close:w }

```

¶ By default, the list contains only one item.

```

354 { \BNVS_scan:w }
355 }

```

1.3.7 ISR unit

Here is a partial grammar for identifiers,

```

2  <ISR>      ::= ( <IS> | <dot_P> ) <R>*
3  <IS>       ::= ( <I>? '!' )? <S>
4  <I>        ::= <S>
5  <S>        ::= <alpha> <alnum>*
6  <dot_P>    ::= '.' <P>
7  <P>        ::= <digit>* <S>
8  <R>        ::= <dot_P> | <dot_N> | <n>
9  <dot_N>    ::= '.' <N>
10 <n>        ::= <Sqr_open> <n_expr> <Sqr_close>
11 <Sqr_open> ::= <spc> '[' <spc>
12 <n_expr>   ::= <blnc>
13 <Sqr_close> ::= <spc> ']' <spc>

```

and the associate AST :

```

<IS>* ::= \IS:w { <I> } { <S> }
<ISR>* ::= \ISR:w { <I> } { <S> } { <R>* }
<R>* ::= <none>
        | \P:nn { <P> } { <R>* }
        | \N:nn { <N> } { <R>* }
        | \n:nn { <n_expr> } { <R>* }

```

\c__bnvs_IKS_regex The start of a qualified dotted name, with three capture groups.

(End of definition for \c__bnvs_IKS_regex.)

```

356 \regex_const:Nn \c__bnvs_IKS_regex {
    1: the optional frame <id> with
    2: the optional exclamation mark
357   (? : ( \ur{c__bnvs_S_regex} )? ( ! ) )?
    3: followed by a non empty short name.
358   ( \ur{c__bnvs_S_regex} )
359 }

```

\BNVS_scanned_IKS:nw \BNVS_scanned_IKS:nw {<scan:w>} {<I>} {<K>} {<S>} <input stream>

Only called by \BNVS_scan_IS:TFw. Calls \BNVS_scan_R:nw.

```

360 \cs_new:Npn \BNVS_scanned_IKS:nw #1 #2 #3 #4 {
361   \tl_if_empty:NTF { #3 } {
    362     \__bnvs_tl_clear_new:c { S( \BNVS_tl_use:c I ) }
    363     \__bnvs_tl_set:cn { S( \BNVS_tl_use:c I ) } { #4 }
    364   } {
    365     \__bnvs_tl_set:cn I { #2 }
    366     \__bnvs_tl_clear_new:c { S(#2) }
    367     \__bnvs_tl_set:cn { S(#2) } { #4 }
    368   }
    369   \__bnvs_tl_set:cn S { #4 }

```

❗ No frame identifier given, use the current value. Store the short name, store in the appropriate variable.

❗ Frame identifier given. Proceed as before mutatis mutandis.

❗ Create a scan group to manage the <prefix:w>.

```

370 \BNVS_scan:nnw {

```

❗ By default we start on a standalone identifier,

```

371   \BNVS_xprt_wrap_prefix:n { \IS:w }
372   \BNVS_xprt_wrap:n { ##1 }
373 } {

```

❗ Export <I> and <S>.

```

374   \BNVS_tl_use:Nv \BNVS_xprt_grp:n I
375   \BNVS_xprt_grp:n { #4 }

```

❗ Execute the <scan:w> instructiony. The <prefix:w> will change.

```

376     #1
377   }
378 }

```

\BNVS_scan_IS:TFw **\BNVS_scan_IS:TFw** {<yes:w>} {<no:w>} <input stream>

Scan an <IS> unit if any, stores the <I>, <S>, and <R> components, between braces, and execute <yes:w> instructions. The <no:w> instructions are executed when no <IS> unit is found. The arguments are expected to call the appropriate branch function.

```

379 \cs_new:Npn \BNVS_scan_IS:TFw #1 #2 {
380   \peek_regex_replace_once:NnTF \c__bnvs_IKS_regex { { \1 } { \2 } { \3 } } {
381     \BNVS_scanned_IKS:nw { #1 }
382   } {

```

❗ Choose the false branch.

```

383     #2
384   }
385 }

```

\BNVS_scanned_dot_P:nw **\BNVS_scanned_dot_P:nw** {<scan:w>} {<I>} {<K>} {<S>} <input stream>

Only called by **\BNVS_scan_dot_P:TFw**.

```

386 \tl_new:N \l__bnvs_other_X_tl
387 \tl_set_rescan:Nnn \l__bnvs_other_X_tl {
388   \char_set_catcode_other:N \X
389 } { X }
390 \cs_new:Npn \BNVS_scanned_dot_P:nw #1 #2 {

```

❗ No frame identifier given, no short name given.

```

391   \__bnvs_tl_if_exist:cF { S( \BNVS_tl_use:c I ) } {

```

It defaults to X.

```

392     \BNVS_tl_new:c { S( \BNVS_tl_use:c I ) }
393     \__bnvs_tl_set:cv { S( \BNVS_tl_use:c I ) } { other_X }
394   }
395   \__bnvs_tl_set:cv S { S( \BNVS_tl_use:c I ) }

```

❗ Create a scan group to manage the <prefix:w>.

```

396   \BNVS_scan:nw {

```

❗ By default we start on a standalone identifier,

```

397     \BNVS_xprt_wrap_prefix:n { \ISR:w }
398     \exp_args:Ne \BNVS_xprt_wrap_alt:n {
399       { \__bnvs_tl_use:c I }
400       { \__bnvs_tl_use:c S }
401       { \exp_not:N \P:nn { #2 } \exp_not:n { { ##1 } } }
402     }
403   } {

```

❗ Execute the <scan:w> instructions. The <prefix:w> will most certainly change.

```

404     #1
405   }
406 }

```

\BNVS_scan_dot_P:TFw **\BNVS_scan_dot_P:TFw** {<yes:w>} {<no:w>} <input stream>

Scan a <dot_P> unit and execute <yes:w> instructions. The <no:w> instructions are executed when no <dot_P> unit is found. The arguments are expected to call the appropriate branch function.

```

407 \cs_new:Npn \BNVS_scan_dot_P:TFw #1 #2 {
408   \peek_charcode_remove:NTF . {
409     \peek_regex_replace_once:NnTF \c__bnvs_P_regex { { \0 } } {
410       \BNVS_scanned_dot_P:nww { #1 }
411     } {

```

❏ Choose the false branch and reinject the dot.

```

412       #2
413       .
414     }
415   } {

```

❏ Choose also the false branch.

```

416     #2
417   }
418 }

```

\BNVS_scanned_P:nww **\BNVS_scanned_P:nww** {<scan:w>} {<P>} <input stream>

Only called by **\BNVS_scan_R:TFw**.

```

419 \cs_new:Npn \BNVS_scanned_P:nww #1 #2 {
420   \BNVS_scan:nnw {
421     \BNVS_xprt_wrap_alt:n { \PR:w { #2 } { ##1 } }
422   } {
423     \BNVS_scanned_alter_set:n { \BNVS_end_scan: #1 }
424     \BNVS_scan_R:TFw { \BNVS_scanned_alter:w } { \BNVS_scanned_alter:w }
425   }
426 }

```

\BNVS_scanned_N:nww **\BNVS_scanned_N:nww** {<scan:w>} {<N>} <input stream>

Only called by **\BNVS_scan_R:TFw**.

```

427 \cs_new:Npn \BNVS_scanned_N:nww #1 #2 {
428   \BNVS_scan:nnw {
429     \BNVS_xprt_wrap_alt:n { \NR:w { #2 } { ##1 } }
430   } {
431     \BNVS_scanned_alter_set:n { \BNVS_end_scan: #1 }
432     \BNVS_scan_R:TFw { \BNVS_scanned_alter:w } { \BNVS_scanned_alter:w }
433   }
434 }

```

\BNVS_scanned_n:nw **\BNVS_scanned_n:nw** {<scan:w>} <input stream>

Only called by **\BNVS_scan_R:TFw**.

```

435 \cs_new:Npn \BNVS_scanned_n:nw #1 {
436   \BNVS_scan_close:nnnNnw {
437     \BNVS_xprt_wrap_alt:n { \nR:w { ##1 } }
438   } {

```


¶ On close branch, new group to scan $\langle R \rangle$ after $\langle n \rangle$.

```

439     \BNVS_scan:nnw {
440         \BNVS_xprt_wrap_grp:n { ###1 }
441     } {
442         \BNVS_scanned_alter_set:n { \BNVS_end_scan: #1 }
443         \BNVS_scan_R:TFw { \BNVS_scanned_alter:w } { \BNVS_scanned_alter:w }
444     }
445 } {
446     \BNVS_end_scan: \BNVS_scanned_stop:
447 } %[
448 ] {
449     \BNVS_scan_n_expr:nnw { \BNVS_scanned_close:w } { \BNVS_scanned_stop: }
450 }
451 }
```

\BNVS_scan_R:TFw \BNVS_scan_R:TFw { $\langle yes:w \rangle$ } { $\langle no:w \rangle$ } $\langle input\ stream \rangle$

Scan an $\langle R \rangle$ and execute $\langle yes:w \rangle$ instructions, which must send `\BNVS_scanned_alter:w` or `\BNVS_scanned_close:w` *in fine*. The $\langle no:w \rangle$ instructions are executed when no $\langle R \rangle$ is found. The arguments are expected to call the appropriate branch function.

```

452 \cs_new:Npn \BNVS_scan_R:TFw #1 #2 {
453     \peek_charcode_remove:NTF . {
454         \peek_regex_replace_once:NnTF \c__bnvs_P_regex { { \0 } } {
455             \BNVS_scanned_P:nww { #1 }
456         } {
457             \peek_regex_replace_once:NnTF \c__bnvs_N_regex { { \0 } } {
458                 \BNVS_scanned_N:nww { #1 }
459             } {
```

¶ Choose the false branch and reinject the dot.

```

460         #2
461         .
462     }
463 }
464 } {
465     \peek_regex_replace_once:NnTF \c__bnvs_Sqr_open_regex { } {
466         \BNVS_scanned_n:nw { #1 }
467     } {
```

¶ Choose also the false branch.

```

468     #2
469     }
470 }
471 }
```

1.3.8 $\langle Rnd \rangle$, $\langle Rslv \rangle$ and $\langle ISRC \rangle$ units

Here is the grammar

```

7   $\langle Rnd(\langle scan:w \rangle) \rangle ::= \langle Rnd\_open \rangle \langle Rnd\_csl(\langle scan:w \rangle) \rangle \langle Rnd\_close \rangle$ 
8   $\langle Rnd\_open \rangle ::= \langle spc \rangle '(' \langle spc \rangle$ 
9   $\langle Rnd\_csl \rangle ::= \langle in\ progress \rangle$ 
10  $\langle Rnd\_close \rangle ::= \langle spc \rangle ')' \langle spc \rangle$ 
11  $\langle Rslv \rangle ::= '?' \langle Rnd \rangle$ 
12  $\langle ISRC \rangle ::= \langle ISR \rangle \langle C \rangle$ 
13  $\langle C \rangle ::= \langle Rnd \rangle$ 

```

with the associate AST

```

 $\langle ISRC \rangle^*$       ::=  $\backslash ISRC:w \{ \langle I \rangle \} \{ \langle S \rangle \} \{ \langle R \rangle^* \} \{ \langle Rnd\_csl \rangle^* \}$ 
 $\langle Rslv \rangle^*$      ::=  $\backslash Rslv:n \{ \langle Rnd\_csl \rangle^* \}$ 

```

```

\BNVS_scan_Rnd:TFw \BNVS_scan_Rnd:nTTnFw { $\langle Rnd\ preamble: \rangle$ } { $\langle close:w \rangle$ } { $\langle stop: \rangle$ } { $\langle scan\ preamble: \rangle$ }
{ $\langle no:w \rangle$ }  $\langle input\ stream \rangle$ 

```

Scan a $\langle Rnd \rangle$ unit if any and execute one of $\langle close:w \rangle$ or $\langle stop: \rangle$ instructions, otherwise execute $\langle no:w \rangle$ instructions.

```

472 \cs_new:Npn \BNVS_scan_Rnd:nTTnFw #1 #2 #3 #4 #5 {
473   \peek_regex_replace_once:NnTF \c__bnvs_Rnd_open_regex { } {
474     \BNVS_scan_close:nnnNnw {
475       \BNVS_xprt_wrap_grp:n { ##1 }
476       #1
477     } { #2 } { #3 } %(
478   ) {
479     \BNVS_scan_csl:nw { #4 }
480   }
481 } {
482   #5
483 }
484 }

```

```

\BNVS_scan_Rslv:nnFw \BNVS_scan_Rslv:nnFw { $\langle close:w \rangle$ } { $\langle stop: \rangle$ } { $\langle no:w \rangle$ }  $\langle input\ stream \rangle$ 

```

Scan a $\langle Rslv \rangle$ unit, if any, and execute one of $\langle close:w \rangle$ or $\langle stop:w \rangle$ instructions, otherwise execute $\langle no:w \rangle$ instructions.

```

485 \cs_new:Npn \BNVS_scan_Rslv:nnFw #1 #2 #3 {
486   \peek_charcode_remove:NnTF ? {
487     \BNVS_scan_Rnd:nTTnFw
488     { \BNVS_xprt_wrap_prefix:n { \Rslv:n } }
489     { #1 } { #2 } { } { #3 ? }
490   } {
491     #3
492   }
493 }

```

```
\BNVS_scan_C:nnFw \BNVS_scan_C:nnFw {<close:w>} {<stop:w>} {<no:w>} <input stream>
```

Scan a $\langle C \rangle$ unit, if any, and execute one of $\langle close:w \rangle$ or $\langle stop:w \rangle$ instructions, otherwise execute $\langle no:w \rangle$ instructions.

```
494 \cs_new:Npn \BNVS_scan_C:nnFw #1 #2 #3 {
495   \BNVS_scan_Rnd:nTTnFw { } { #1 } { #2 } { } { #3 }
496 }
```

1.3.9 Call and assignment units

Here is the grammar

```
3 <ISRCME> ::= <ISR> | <ISRC> | <ISRME>
4 <ISRC>    ::= <ISR> <Rnd>
5 <ISRME>   ::= <ISR> <ModEq> <Rhs>
6 <ModEq>   ::= <spc> ( <Mod> '=' ) <spc>
7 <Mod>     ::= ' ' | '+' | '-' | '/' | '*' |
8 <Rhs>     ::= <VAZLRhs> | <Rslv> | <Raw>
9 <VAZLRhs> ::= <VAZL> | <VAZLRnd> | <Crl> | <Sqr>
10 <Rslv>    ::= '?' <Rnd>
```

with the associate AST

```
<ISRC>* ::= \ISRC:w { <I> } { <S> } { <R>* } { <Rnd_csl>* }
<ISRME>* ::= \ISRME:w { <I> } { <S> } { <R>* } { <Mod> } { <Rhs>* }
```